-- coding: utf-8 --

本报告由 QuantMind 投研分析管线自动生成
仅供学习研究参考，不构成任何投资建议

本文件由 QuantMind 自动生成，仅供学习研究与技术演示，不构成任何投资建议。

QuantMind 策略编写与回测教程

> 版本：2026-10-26

> 适用：QuantMind OSS 版（A 股为主，架构支持港股/美股/加密/期货）

> 读者：策略研究员、量化开发、AI-IDE 用户

> 说明：本文所有参数均按 2026-10-26 代码实际 schema 整理；界面字段可能随版本微调，以页面提示为准。

> **免责声明**：本教程仅供学习研究与技术演示，不构成任何投资建议。回测收益不代表未来表现；实盘前请完成模拟盘验证，并咨询持牌专业机构。股市有风险，投资需谨慎。

0. 先选路径：你要写哪种策略?

QuantMind 里“策略”不是一个东西，而是**五种代码形态**。先按目标选形态，能少走很多弯路：

你的目标	推荐形态	在哪里写	能否直接进策略库	回测引擎
用平台模型预测分做 Top-K 选股	`STRATEGY_CONFIG` 声明式	策略管理 / 回测中心 / AI-IDE	☒ 可以	Qlib
自己写均线、通道、量价等脚本策略	minibt DSL	AI-IDE / 回测中心	☒ 可以	minibt（A 股日线）
盘中逐 Tick 驱动、对接模拟盘	`on_tick(context)` 沙箱	模拟盘上传 / AI-IDE	⚠ 运行时形态，不走策略库标准格式	沙箱模拟盘
像写 Jupyter 一样快速试想法	Strategy Lab `setup/on_bar`	策略实验室	☒ 可“转模板”后进库	Qlib
临时跑一段 Python、看日志	AI-IDE 脚本	AI-IDE	⚠ 运行时，不保存为策略	AI-IDE 运行器

一句话选择：

- 有模型、要选股组合 → 用 `STRATEGY_CONFIG`；
- 有明确技术指标规则、单标的研究 → 用 minibt；
- 要盘中事件驱动、跑模拟盘 → 用 `on_tick`；
- 要快速验证想法、做过拟合体检 → 用 Strategy Lab；
- 只是调试代码 → 用 AI-IDE。

> 已下架：聚宽风格 `initialize/handle_data`。系统没有它的执行器，不要再用这种写法。

1. 平台地图：策略、回测、模拟盘的关系

```
策略管理（策略库）
├── 保存 STRATEGY_CONFIG / minibt 代码 + 参数 + 版本 + 状态
├── 回测中心：快速回测 / 专家模式 / 参数优化 / 策略对比 / 历史 / 高级分析
└── 模拟盘：选库内策略或上传文件 → 按交易配置运行
```

```
AI-IDE
├── 写代码 / 调试
├── minibt 代码自动切专用运行时
├── 模块型策略（STRATEGY_CONFIG）走“回测中心兼容模式”
└── 结果 / 日志 / 回测结果文件

策略实验室 Strategy Lab
├── setup/on_bar 快速回测
├── 4 关卡过拟合检测
├── 转模板 → 进策略库
└── 每日扫描（可选）

模拟盘
├── 沙箱 on_tick / STRATEGY_CONFIG 策略
├── minibt 策略（按日评估，单标的）
└── T+1、涨跌停、整手、费用、台账
```

关键边界：

1. 回测中心默认用 **Qlib 引擎**；minibt 代码会被识别并转到专用运行时，不能直接进 Qlib。

2. 策略库只保留两种可执行格式：`STRATEGY_CONFIG`、minibt。`on_tick`、AI-IDE 脚本是**运行时形态**。

3. 回测不是“看完就算”：回测完成后会生成**九项体检**，A/B 才能晋级模拟盘，L/E 不能晋级（详见 §10）。

4. 模拟盘和回测是两套成交口径：模拟盘有 T+1、涨跌停、整手、费用；minibt 回测没有 A 股规则，见 §3.4。

1.4 策略编辑（回测中心）的右栏

「回测中心 → 策略编辑」右侧是**回测工作台**，不是聊天窗口：

区块	作用
参数调整	标的/股票池/起止日期/初始资金/佣金/topk/n_drop/调仓周期/SIZE/快慢线
仅应用参数	把参数写回代码（minibt 改顶层常量与 params；STRATEGY_CONFIG 改 kwargs）
应用并运行	写回代码后立即跑回测
本次结果	净值、K 线买卖点、月度热力、成交、持仓等专业图表
保存本次	把当前结果存到本机浏览器（最多 20 条，可加载/删除；正式台账仍看回测历史）
AI 写策略	跳转到 **QuantBot** 写策略；策略编辑不再内嵌 AI 对话

> AI-IDE 独立页面（`/ai-ide`）仍保留内置 AI 助手；回测中心的策略编辑只做“参数 + 回测 + 图表 + 存档”。

2. 写法一：Qlib 声明式 `STRATEGY_CONFIG`

这是策略库、回测中心、模拟盘最常用的格式：**不写引擎代码，只声明策略类和参数**。

2.1 最小示例（标准 Top-K 选股）

```
# -*- coding: utf-8 -*-
"""标准 Top-K 选股策略。"""
STRATEGY_CONFIG = {
"class": "RedisTopkStrategy",
"kwargs": {
"signal": "<PRED>", # 使用模型预测分；配合回测请求里的 model_id
"topk": 50, # 持仓股票数
"n_drop": 10, # 每期最多替换数
},
}
```

- `class`：平台内置策略类（见 ⊠2.3）；
- `module_path`：可选；内置类可省略，平台会自动补全路径；
- `kwargs`：传给策略类构造函数的参数；
- `signal="<PRED>"`：占位符，表示用回测请求里 `model_id` 对应模型的 `pred`；也可以传自定义信号文件路径。

2.2 结构说明

```
STRATEGY_CONFIG = {
"class": "RedisRecordingStrategy", # 必填：策略类名
"module_path": "backend.services.engine.qlib_app.utils.recording_strategy", # 可选
"kwargs": { # 必填：策略参数
"signal": "<PRED>",
"topk": 30,
"n_drop": 6,
"rebalance_days": 5,
# 其余参数见下文字典
},
}
```

参数优先级（模板模式）：

- 通过回测中心/模板界面调整的、模板 JSON 里声明过的参数，**界面值优先**；
- 模板未声明的 `kwargs`，**代码值优先**；
- 非模板的自定义策略，代码里的 `kwargs` 优先，界面只覆盖已声明参数。

> 写模板时，把希望用户可调的参数放进模板 JSON 的 `params`；否则用户在界面上改不动。

2.3 常用策略类与核心参数

策略类	用途	核心参数（除 signal 外）
`RedisTopkStrategy`	标准 Top-K + 零换手约束	`topk`、`n_drop`、`rebalance_days`
`RedisRecordingStrategy`	通用 Top-K + 基本面过滤 + 动态仓位	`topk`、`n_drop`、`rebalance_days`、`f_*`、`risk_degree`、`max_leverage`、`account_stop_loss`
`RedisWeightStrategy`	按预测分/权重分配资金	`topk`、`min_score`、`max_weight`
`RedisLongShortTopkStrategy`	多空 Top-K 对冲	`topk`、`short_topk`、`long_exposure`、`short_exposure`、`min_score`、`max_weight`、`enable_short_selling`
`RedisRiskGuardTopkStrategy`	Top-K + 行业/市场状态风控	`max_industry_count`、`industry_cap_ratio`、`market_state_window`、`f_*`
`RedisMomentumStrategy`	动量增强	`momentum_period`、`momentum_weight`
`RedisAdvancedAlphaStrategy`	截面 Alpha 加权	`min_score`、`max_weight`
`RedisSectorRotationStrategy`	行业轮动	`topk_sectors`、`lookback_days`
`RedisStopLossStrategy`	硬止损/止盈	`stop_loss`、`take_profit`
`RedisCrashBuyDipStrategy`	大盘急跌抄底	`crash_threshold_points`、`crash_threshold_pct`、`top_k`、`hold_days`、`take_profit`、`stop_loss`、`benchmark`、`trend_window`、`ma_fast`、`ma_slow`、`vol_lookback`

通用回测参数（`QlibStrategyParams`，界面/请求字段）：

参数	默认	范围	说明
`topk`	50	5⊠200	选股数量
`short_topk`	50	0⊠200	做空数量（多空策略）
`n_drop`	10	0⊠200	每期最大换手数
`signal`	`<PRED>`	—	信号列/文件；`<PRED>` 用模型预测
`min_score`	0.0	⩾0	权重策略最低分
`max_weight`	1.0	0⊠1	单票最大权重
`long_exposure`	1.0	0⊠3	多头敞口
`short_exposure`	1.0	0⊠3	空头敞口
`rebalance_days`	3	1⊠60	回测调仓周期（与模拟盘的 1/3/5/10/20 不同！）
`momentum_period`	20	5⊠60	动量周期
`topk_sectors`	5	2⊠15	行业轮动选几个行业
`lookback_days`	20	5⊠252	回看周期
`vol_lookback`	20	5⊠120	波动率回看
`stop_loss`	-0.08	-0.5⊠-0.01	策略层止损（区别于执行层止损）
`take_profit`	0.15	0.01⊠1.0	策略层止盈
`enable_short_selling`	false	—	是否做空
`financing_rate`	0.08	0⊠1	融资年化
`borrow_rate`	0.08	0⊠1	融券年化
`max_short_exposure`	1.0	0⊠3	最大空头敞口
`max_leverage`	1.0	0⊠5	最大总杠杆
`account_stop_loss`	0.2	0⊠0.8	账户级止损；0 表示关闭

> 说明：`topk`、`n_drop`、`rebalance_days` 等字段是**回测请求**的通用参数；具体策略类可能只消费其中一部分，模板里写的 `kwargs` 才是最终值。平台会把不认识的参数过滤掉并在日志里提示。

2.4 基本面硬过滤：`f_*` 参数

在 `kwargs` 里写 `f_` 前缀，即可在选股前做**硬过滤**。过滤使用**上一交易日**的基本面快照，避免前视偏差。

写法	含义	操作符
`f_pe_ttm_max`	PE ⩽ 值	`<=`
`f_pe_ttm_min`	PE ⩾ 值	`>=`
`f_is_st_not`	ST 状态 ≠ 值	`!=`
`f_industry_in`	行业属于集合	`in`
`f_roe`	ROE == 值	`==`

常用字段（完整 88 维见《实盘交易因子筛选手册》）：

类别	字段
基础	`listed_days`、`is_st`、`industry`、`listing_market`
估值	`pe_ttm`、`pb`、`total_mv`、`float_mv`、`roe`
收益动量	`return_5d`、`return_20d`、`ma_gap_20`
技术	`rsi_6`、`vol_std_20`、`beta_20`
量能资金	`turnover_rate`、`volume_ratio_5`、`inst_ownership`、`amount_ma_5`
指数状态	`idx_hs300`、`idx_zz1000`、`limit_up_today`

单位注意：

- `roe`、`return_20d`、`vol_std_20` 等是**比率**：0.12 = 12%；
- `total_mv`、`float_mv` 是**元**：30 亿写 `3e9`；
- `amount_ma_5` 是**万元**：5000 = 5000 万元；
- `pe_ttm` 缺失的标的会被过滤；想保留亏损股就不要写 `f_pe_ttm_min`。

示例：

```
STRATEGY_CONFIG = {
"class": "RedisRecordingStrategy",
"kwargs": {
"signal": "<PRED>",
"topk": 30,
"n_drop": 5,
"rebalance_days": 5,
"f_is_st_not": 1,
"f_total_mv_min": 3e9,
"f_total_mv_max": 5e10,
"f_amount_ma_5_min": 5000,
"f_pe_ttm_min": 0,
"f_pe_ttm_max": 60,
"f_roe_min": 0.10,
"f_listed_days_min": 365,
},
}
```

2.5 动态仓位与账户风控

参数	说明
`risk_degree`	默认仓位系数（0⊠1，按 max_leverage 夹紧）
`max_leverage`	最大总杠杆（0⊠5）
`account_stop_loss`	账户级止损：0 关闭；0.8 表示净值跌破初始 80% 平仓；-0.2 等价
`market_state_symbol`	市场状态参考指数（如 SH000300）
`market_state_window`	状态窗口（交易日）
`position_by_state`	状态 → 仓位映射，如 `{"bull": 1.0, "neutral": 0.6, "bear": 0.2}`
`strategy_total_position`	策略总资金占比（0⊠1）
`dynamic_position`	回测请求开关：启用动态仓位

> 动态仓位是**降低回撤**的工具，不是提高收益的保证。开启后必须回测对比“开/关”两组，确认不是靠某一段行情。

2.6 保存、版本与状态

- 策略管理保存时会记录：代码、参数、`strategy_type`、版本、`code_hash`、状态；
- 状态建议按 `草稿 → 已验证 → 模拟中 → 实盘中 → 归档` 管理；
- 用户策略启动模拟盘要求状态 **VERIFIED**（回测验证），并通过回测体检 A/B；
- 系统模板（`sys_`）和文件上传策略不走库内状态机，但仍建议先回测。

3. 写法二：minibt DSL（脚本策略）

minibt 是“约 20 行写一个策略”的脚本框架，适合单标的技术指标研究。QuantMind 已内置 **1.3.3**，运行在专用 runner 容器（Python 3.12）。

3.1 最小示例：双均线

```
# -*- coding: utf-8 -*-
"""双均线：金叉做多，死叉平仓。"""
from minibt import Bt, Strategy
from backend.shared.minibt_qdb import load_daily
from backend.shared.minibt_result import run_and_report

SYMBOL = "600036"
START = "20240101"
END = "20261026"
SIZE = 1000 # 每次交易股数
COMMISSION = 0.00025 # 双边佣金

DF = load_daily(SYMBOL, START, END)

class DualMA(Strategy):
params = dict(fast=10, slow=20)

def __init__(self):
self.kline = self.get_kline(DF, duration_seconds=86400)
```

```
self.percent_commission = COMMISSION
self.ma_fast = self.kline.close.sma(self.params.fast)
self.ma_slow = self.kline.close.sma(self.params.slow)
self.golden = self.ma_fast.cross_up(self.ma_slow)
self.death = self.ma_fast.cross_down(self.ma_slow)

def next(self):
if not self.kline.position:
if self.golden.new:
self.kline.buy(size=SIZE)
elif self.death.new:
self.kline.sell(size=SIZE)
elif self.kline.position > 0 and self.death.new:
self.kline.sell(size=SIZE)
elif self.kline.position < 0 and self.golden.new:
self.kline.buy(size=SIZE)

if __name__ == "__main__":
bt = Bt(auto=False)
bt.addstrategy(DualMA)
run_and_report(bt, DF)
```

3.2 数据与常用 API

API	用途
`load_daily(symbol, start, end)`	读 QuantDB A 股前复权日线
`Bt(auto=False)`	创建回测实例
`bt.addstrategy(类)`	添加策略
`self.get_kline(DF, duration_seconds=86400)`	绑定日线
`self.kline.close.sma(n)` / `.ema(n)`	指标
`cross_up` / `cross_down` / `.new`	交叉信号，`.new` 只触发一次
`self.kline.buy(size=股数, stop=...)`	买入；`sell` 同理
`self.kline.position`	当前方向（1/0/-1）
`self.percent_commission`	佣金比例（默认 0，必须显式设）
`BtStop.SegmentationTracking`	分段跟踪止损
`run_and_report(bt, DF)`	执行并输出结构化结果

`Strategy.params` 是任意字典，界面上可通过模板 `params` 暴露成参数；常用：

参数	说明
`fast`、`slow`	均线周期
`SIZE`	每次交易股数（100 的整数倍）
`COMMISSION`	佣金比例
其余	按策略自行定义

3.3 minibt 的回测口径（重要）

项	口径
成交价	信号当根 K 线**收盘价**成交（默认 `OrderType.Close`）；`OrderType.Market` 才是下根开盘
A 股规则	**不建模** T+1、涨跌停、整手、ST
手续费	默认 0；必须显式 `percent_commission`
收益口径	自带报告“最终收益”不含未平仓浮盈；平台用 `run_and_report` 修正
反手	持多时 `sell()` 只平不开空；反手要显式两步
做空	回测支持，但 A 股现货不能裸卖空；模拟盘会把负目标按 0 处理
桌面图表	服务端一律禁用，`run(isplot=False, isreport=False)`

> 因此：**minibt 回测结果不能直接当 A 股实盘预期**。它适合验证“指标逻辑是否成立”，不适合直接评估 T+1/涨跌停后的收益。

3.4 在 AI-IDE / 回测中心跑 minibt

- 代码行首出现 `import minibt` 或 `from minibt import ...`，AI-IDE 会自动切到专用 runner；
- 回测中心也支持派发 minibt 策略，结果会映射成与 Qlib 同构的指标/逐笔/权益；
- Qlib 回测入口会明确拒绝 minibt 代码，并提示“请在 AI-IDE 使用 minibt 运行时”。

3.5 minibt 接入模拟盘

- 策略库里保存 minibt 代码（`strategy_type` 以 `minibt_` 开头）后，可启动模拟盘；
- 模拟盘按**日线、单标的**评估：买入窗口前预计算目标股数，14:50 附近执行；
- 目标股数只作用在该策略的标的上，不会动账户里其他持仓；
- 目前只支持 **A 股、日线、单标的、模拟盘**；实盘会明确拒绝，多标的/分钟线待扩展。

3.6 1.3.3 新能力

- Pine 风格指标库、ACP 自动形态、Volume Profile 等新指标可直接在 minibt 策略里用；
- `signal_backtest(gui=...)`、`from_signals` 逐笔统计修复；
- 回测速度比 1.2.7 慢约 6 倍（约 2 秒/2595 根日线），对“每日一次决策”的模拟盘无影响。

4. 写法三：沙箱 `on_tick`（模拟盘/实盘）

这是模拟盘最传统的策略形态：系统每 10 秒调用一次 `on_tick(context)`，策略主动下单。

4.1 最小示例

```
# -*- coding: utf-8 -*-
"""每分钟检查一次：空仓买 1000 股，持有一段时间后卖出。"""
def on_tick(context):
symbol = "SH600036"
pos = context.get_position(symbol)
if pos["volume"] == 0:
context.order_target_percent(symbol, 0.2) # 目标 20% 仓位
context.log("建仓 20%")
elif pos["volume"] > 0:
# 这里只是示例：实际请加入你自己的卖出条件
pass
```

4.2 Context API

API	说明
`order_target_percent(symbol, target_percent)`	目标仓位比例；系统按账户总资产计算股数
`order(symbol, quantity, price, side, order_type="limit")`	直接下单
`get_position(symbol)`	返回 `volume`、`available_volume`、`cost`、`price`、`market_value`
`get_cash()`	可用现金
`get_total_asset()`	总资产
`log(message)`	写入模拟盘运行日志

4.3 交易配置

`on_tick` 策略通常配合前端“模拟盘托管”配置：

- 调仓节奏、执行时段、买卖时间、先卖后买、订单类型、价格偏离、单轮委托上限；
- 这些配置在 `live_trade_config` 里，详见 ⌧9；
- 代码里也可以写 `STRATEGY_CONFIG = {...}` 由平台解析交易参数覆盖（与 Qlib 策略的字段同名）。

4.4 限制

- 沙箱只认 `on_tick` / `STRATEGY_CONFIG` 形态；**minibt 代码会被明确拒绝**（避免静默零信号）；
- 用户代码在 AST 白名单内运行，禁止 `os`、`sys`、`subprocess`、网络、文件写入等；
- 模拟盘有 T+1、涨跌停、整手、费用、参与率等真实约束，回测与模拟盘结果会有差异；
- 实盘同样走这套沙箱，但需通过实盘双开关、风控和确认链。

5. 写法四：Strategy Lab（快速验证 + 过拟合体检）

Strategy Lab 是“浏览器版 Jupyter + 回测 + K 线”的快速研究环境，用 `setup/on_bar` 钩子，不直接暴露 Qlib 内部对象。

5.1 最小示例

```
# -*- coding: utf-8 -*-
"""双均线：5 日线上穿 20 日线买入，下穿卖出。"""
def setup(ctx):
ctx.universe = "csi300"
ctx.start = "2024-01-01"
ctx.end = "2025-12-31"
ctx.cash = 1_000_000
ctx.benchmark = "SH000300"

def on_bar(ctx, bar):
closes = ctx.history(bar.symbol, n=20, field="close")
if len(closes) < 20:
return
fast = closes.iloc[-5:].mean()
slow = closes.mean()
pos = ctx.position(bar.symbol)
if fast > slow and pos.qty == 0:
ctx.buy(bar.symbol, weight=0.05, reason="MA5>MA20")
elif fast < slow and pos.qty > 0:
ctx.sell(bar.symbol, all=True, reason="MA5<MA20")
```

5.2 `ctx` 配置参数

属性	默认	说明
`universe`	必填	内置池或自定义列表；内置池见下
`stock_pool`	None	全局股票池引用（如 `pool:csi1000`），与 universe 取交集
`start` / `end`	必填	YYYY-MM-DD
`cash`	必填	初始资金
`benchmark`	SH000300	基准
`commission`	0.0003	佣金
`slippage`	0.0005	滑点
`tax_sell`	0.0005	卖出印花税
`transfer_fee`	0.00001	过户费
`execution_model`	a_share_strict	`a_share_strict` / `simple`
`max_position_per_stock`	0.10	单票上限
`max_positions`	10	最大持仓数
`engine`	qlib	目前仅 Qlib
`freq`	day	日频

内置股票池：`csi300`、`csi500`、`csi1000`、`sse50`、`gem`、`star`、`csi800`、`all_a`、`hs300_ext`、`hk_main`、`us_sp500`。港股/美股池依赖对应数据接入，未就绪时返回空并给出警告。

5.3 常用 `ctx` API

类别	API
数据	`history(symbol, n, field)`、`feature(symbol, name, n)`、`list_features()`、`snapshot(date)`、`benchmark_history(n)`
下单	`buy(symbol, weight/qty, reason, detail)`、`sell(symbol, weight/all, reason)`、`set_position`、`set_target_holdings`
风控	`set_stop_loss`、`set_take_profit`、`set_account_stop_loss`、`set_max_holding_days`
查询	`position(symbol)`、`positions()`、`cash`、`equity`、`today`
工具	`log`、`plot_line`、`plot_marker`、`drawn_line`、`param`、`is_st`、`is_tradable`、`industry`、`market_cap`

5.4 4 关卡过拟合检测

回测完成后可点“4 关卡检测”：

1. **训测分割**：70/30 时间切分，看 OOS Sharpe 是否塌陷；
2. **3 段走查**：50→70%、70→85%、85→100% 三个 OOS 窗；
3. **参敏扫描**：枚举 `ctx.param()` 取值，看 Sharpe 标准差；
4. **蒙特卡洛**：随机洗牌日收益 100 次，看累计收益排名。

总分 = 0.3×关卡1 + 0.3×关卡2 + 0.2×关卡3 + 0.2×关卡4，再与基础评分按 4:6 混合。它是“研究阶段”的过拟合体检，和 ☒10 的正式回测体检不是同一套。

5.5 转模板 / 每日扫描

- **转模板**：把 Lab 脚本翻译成 `STRATEGY_CONFIG` 模板，保存到“我的策略”；
- **每日扫描**：注册到 `qm:lab:watch`，工作日 18:30 自动跑一次（需开启 `STRATEGY_LAB_SCAN_ENABLED`）。

6. 写法五：AI-IDE 脚本（运行时）

AI-IDE 适合调试代码，不是策略库格式。支持三种入口：

1. `if __name__ == "__main__":` 或顶层可执行代码；
2. `main()` / `run()` 函数；
3. `STRATEGY_CONFIG` / `get_strategy_config()` / `get_strategy_instance()`（模块型策略兼容模式）。

执行规则：

- 行首 `import minibt` → 切 minibt runner；
- 模块型策略 + `qlib_provider_uri` → 走“回测中心兼容模式”，用 Qlib 引擎回测；
- 其他代码 → 普通 Python 脚本运行；
- 结果通过日志/回测结果文件返回；minibt 结果可在 AI-IDE 的回测面板查看。

注意：

- AI-IDE 的脚本不自动进策略库；要复用请保存为 `STRATEGY_CONFIG` 或 minibt 模板；
- 模块型策略回测会写回测历史/体检表；如遇数据库权限错误，升级到包含 2026-10-26 修复的版本。

7. 回测类型与支持矩阵

7.1 Qlib 回测（回测中心主引擎）

能力	说明
同步回测	`POST /backtest`，适合短周期
异步回测	Celery 队列，适合长周期/大股票池
快速回测	选策略、股票池、日期、初始资金、调仓周期、基准、成交价
专家模式	直接写 `STRATEGY_CONFIG` 代码，支持 `CustomStrategy`
参数优化	网格搜索，最多 100 组；目标指标可选 Sharpe 等
策略对比	多个回测结果并排比较
回测历史	`qlib_backtest_runs` 台账，保留最近 10 条/用户/租户
高级分析	收益/风险/交易/基准/体检证据卡
回测体检	九项检验 → A/B/L/E，见 ☒10

支持市场：

市场	provider_uri / region	说明
A 股	`/data/qlib/cn_data`，region=cn	默认，费用/规则完整
港股	`/data/quanthk/.qlib_cache/hk_data`，region=hk	费用默认按港股口径
美股	`/data/quantus/.qlib_cache/us_data`，region=us	需显式设置费用；默认费用会回落到 A 股口径
加密	`/data/quantbc/.qlib_cache/bc_data`，region=crypto	同上，需显式费用
期货	`/data/quantfutures/.qlib_cache/futures_data`	同上，需显式费用；部分 A 股规则不适用

> 数据目录必须包含 `calendars/day.txt`、`instruments/all.txt`、`features/`；缺一不可。数据未就绪时先到“数据管理/数据平台”同步并构建 Qlib 缓存。

7.2 minibt 回测

能力	说明
运行入口	AI-IDE、回测中心（专用 runner）
市场	A 股日线（QuantDB `daily_forward`）
策略形态	`import minibt` + `Strategy` 子类
结果	`[RESULT]` 标量指标 + `result.json`（权益/逐笔/警告）
规则	无 T+1/涨跌停/整手/ST；信号当根收盘成交
适用	验证指标逻辑、单标的趋势/反转研究

7.3 模块型策略回测

- 策略代码含 `STRATEGY_CONFIG` / `get_strategy_config` / `get_strategy_instance`，且请求带 `qlib_provider_uri`；
- 在 AI-IDE runner 内走 Qlib 回测兼容模式；
- 会写 `qlib_backtest_runs` 与体检 `eval_scores`；
- 普通 AI-IDE 用户代码与它共用 runner，但 DB 权限按运行类型区分（用户代码只读）。

7.4 Strategy Lab 回测

能力	说明
同步/异步	`/strategy-lab/run`、`/run/async`
结果	净值、K 线买卖点、月度热力、成交明细、日志
过拟合	4 关卡（训测/走查/参敏/蒙卡）
转模板	转成 `STRATEGY_CONFIG` 模板
每日扫描	可选，工作日 18:30

7.5 模拟盘（paper trading，不是回测但必须做）

- 策略启动后按 `live_trade_config` 在买入窗口调仓；
- 有 T+1、涨跌停、整手、费用、参与率、风控前置拒单；
- 是回测到实盘之间的**证据层**：用户策略需 VERIFIED + 体检 A/B 才能启动模拟盘；
- minibt 策略按日评估、单标的；沙箱策略按 `on_tick`；Qlib 组合策略按模型信号调仓。

7.6 因子研究（独立栏目）

因子研究/因子回测用于评估单因子 IC、分层收益、换手、衰减等，不是“策略回测”；模型训练后的 T+N 预测再进入 Qlib 回测。

8. Qlib 回测请求参数总表

> 界面“快速回测/专家模式”会映射到这些字段；API 调用时字段名以下表为准。

字段	默认	说明
`strategy_type`	TopkDropout	策略类型/模板 ID
`strategy_params`	—	策略参数对象（见 ⌧2.3）
`strategy_content`	None	自定义策略代码（CustomStrategy）
`model_id`	None	模型 ID；`signal="<PRED>"` 时用它的 pred
`start_date` / `end_date`	近一年	回测区间
`initial_capital`	100000000（请求默认；快速回测界面默认 1000000）	初始资金
`benchmark`	按市场	A 股 SH000300；港股 HSI
`universe`	all	股票池：all/csi300/csi500/csi800/csi1000 等，或自定义
`pool_id`	None	全局股票池引用，优先于 universe
`commission`	按市场	佣金率
`min_commission`	按市场	最低佣金
`stamp_duty`	按市场	印花税（卖出）
`transfer_fee` / `min_transfer_fee`	按市场	过户费
`impact_cost_coefficient`	按市场	冲击成本
`buy_cost` / `sell_cost`	None	综合费率别名，兼容旧接口
`use_vectorized`	false	向量化极速回测
`dynamic_position`	false	动态仓位
`market_state_symbol`	None	市场状态参考指数
`market_state_window`	20	状态窗口
`strategy_total_position`	1.0	策略总资金占比
`deal_price`	open	成交价：open/close
`signal_lag_days`	1	信号滞后交易日；禁止 0 + close（前视）
`risk_free_rate`	0.02	无风险利率
`qlib_provider_uri`	按市场	Qlib 数据目录
`qlib_region`	按市场	cn/hk/us/crypto/futures
`seed`	None	随机种子
`history_source`	manual	manual/optimization（内部）

费用默认值：

市场	佣金	最低佣金	印花税	过户费	冲击成本
A 股	0.00025	5 元	0.0005（卖）	0.00001	0.0005
港股	0.0003	3 HKD	0.001（卖）	0	0.0005
美股/加密/期货	未单独默认	—	—	—	—

> 非 A/港市场请**显式设置**费用；否则会回落到 A 股默认，结果不可比。

9. 执行与模拟盘参数总表

9.1 `execution_config`（执行层风控）

字段	默认	范围	说明
`max_buy_drop`	-0.03	-0.10⊠-0.01	买入时相对参考价的最大跌幅
`stop_loss`	-0.08	-0.20⊠-0.03	执行层止损线

> 注意：这里的 `stop_loss` 是**执行层**风控；策略 `kwargs` 里的 `stop_loss` 是**策略层**止损，两者可以同时存在。

9.2 `live_trade_config`（模拟盘/实盘调度）

字段	默认	说明
`rebalance_days`	5	仅允许 1/3/5/10/20；模拟盘调仓节奏
`schedule_type`	interval	interval/weekly
`trade_weekdays`	[]	weekly 时必填，MON⊠FRI
`enabled_sessions`	[PM]	AM/PM/AFTER_HOURS/NIGHT
`sell_time`	14:30	HH:MM；必须早于 buy_time
`buy_time`	14:45	HH:MM
`sell_first`	true	先卖后买
`order_type`	LIMIT	LIMIT/MARKET
`max_price_deviation`	0.02	限价单最大偏离，0⊠0.05
`max_orders_per_cycle`	20	单轮委托上限，1⊠100
`trigger_window_seconds`	90	触发窗口（调度器实际使用）
`pool_id`	None	全局股票池；与信号取交集
`market`	CN	CN/HK/US/CRYPTO/FUTURES

校验规则：

- `sell_time < buy_time`；
- `schedule_type=interval` 时 `rebalance_days` 必填；
- `schedule_type=weekly` 时 `trade_weekdays` 必填；
- `order_type=LIMIT` 时 `max_price_deviation` 必填；

- 模拟盘托管只在买入窗口跑整轮（卖+买原子执行），卖出窗口单独跳过。

> 回测的 `rebalance_days` 范围是 1□60；模拟盘只允许 1/3/5/10/20。不要把两套值混用。

10. 回测体检与晋级门禁

10.1 九项体检

回测完成后自动跑：

1. PSR（概率夏普）；
2. DSR（Deflated Sharpe，考虑参数尝试次数）；
3. MinTRL（最小可置信样本长度）；
4. Bootstrap（块 bootstrap 收益置信区间）；
5. 收益集中度（剔除最好 N 日后 alpha 是否消失）；
6. 基准回归（alpha/beta/t 值/R□）；
7. 市场状态覆盖（牛/熊/震荡分段是否都活着）；
8. 成本敏感性（换手对应的费用冲击）；
9. PBO/CSCV（参数扫描过拟合概率，有参数矩阵时跑）。

10.2 四分类判定

等级	含义	能否晋级
A	真 alpha（可重复能力）	□ 可晋级
B	Beta 主导（收益主要来自市场/风格）	□ 可晋级，但会标注收益来源
L	运气嫌疑（DSR 低/CI 跨 0/收益集中）	□ 不可晋级
E	证据不足（样本 < MinTRL/缺基准）	□ 不可晋级

判定优先级：**E → L → B → A**。

10.3 门禁

- 用户策略启动模拟盘：状态需 VERIFIED；体检需 A/B；体检曲线至少 60 个点；
- 模拟盘晋级实盘：SIM 运行 ≥20 个交易日；模拟↔真单跟踪误差 ≤0.15（有数据才校验）；
- 系统模板/文件上传策略不受库内状态机约束，但仍建议先回测；
- 体检报告落 `qlib_backtest_runs.result_json.health`；数字 `strategy_id` 同时落 `eval_scores`，供门禁/月度复检读取。

11. 示例库

本教程附带 6 个可运行示例（见同目录 `examples/`）：

文件	形态	用途
`qlib_topk_strategy.py`	STRATEGY_CONFIG	标准 Top-K
`qlib_risk_guard.py`	STRATEGY_CONFIG	风控 + 基本面过滤
`minibt_dual_ma.py`	minibt	双均线
`sandbox_on_tick.py`	on_tick	模拟盘目标仓位
`strategy_lab_dual_ma.py`	Strategy Lab	setup/on_bar 双均线
`ai_ide_script.py`	AI-IDE	普通脚本入口
`live_trade_config.json`	配置	模拟盘调度配置示例

使用前请把 `SYMBOL`、日期、资金、`model_id` 换成你的实际值；示例不构成投资建议。

12. 常见错误与排查

现象	原因	处理
想让 AI 写策略	策略编辑只做参数/回测/图表	点右栏「AI 写策略」跳 QuantBot；AI-IDE 独立页仍有内置助手
点「4 关卡检测/转模板」报 AST 检查未通过	这两个是 Strategy Lab 专用动作，只支持 `setup/on_bar` 脚本；minibt/qlib 结果不应使用	策略编辑右栏已隐藏这些动作；minibt 用回测体检/回测历史
`该策略基于 minibt 脚本框架，qlib 回测引擎不支持`	把 minibt 代码提交到 Qlib 回测	到 AI-IDE 跑，或改用 STRATEGY_CONFIG
回测一直“无信号”	`signal="<PRED>"` 但没传 `model_id`，或模型没有该区间 pred	传 `model_id`；检查预测覆盖到回测区间
`permission denied for table qlib_backtest_runs`	runner 用了只读 DB 角色	升级到 2026-10-26 之后版本（已加模块回测写角色）
模拟盘启动被 409 拒绝	策略未 VERIFIED，或体检不是 A/B	先回测；看体检证据卡；修正策略后重跑
模拟盘不成交	不在交易时段、涨跌停、停牌、T+1 不可卖、价格偏离超限	看运行日志/拒单台账；不要用旧价重试
minibt 回测和实盘差很多	minibt 无 T+1/涨跌停/整手/ST，默认手续费 0	把它当“逻辑验证”；实盘前用模拟盘口径复核
`provider_uri` 不存在	Qlib 缓存未构建/路径不对	数据平台同步并构建缓存；检查 calendars/instruments/features
港股/美股回测费用不对	非 A/港市场费用默认回落 A 股	显式设置 commission/stamp_duty 等
策略界面参数改不动	模板 JSON 没声明该参数	把参数加入模板 `params`，或直接改代码
`signal_lag_days=0 + deal_price=close` 报错	该组合构成前视偏差	用 1+open（标准）或 1+close（尾盘）
minibt runner 不可用	安装/更新时未构建专用镜像	新版安装脚本会自动构建；离线包需自带镜像，或手动 `bash docker/build_minibt_runner.sh`
策略保存后无法回测	代码是空壳/`handle_data` 形态	改成 STRATEGY_CONFIG 或 minibt；`handle_data` 已下架

13. 上线前检查清单

- [ ] 回测区间覆盖牛/熊/震荡，不是只挑一段好行情；
- [ ] 用滚动/样本外验证，不只看全区间收益；
- [ ] 体检等级 A/B，且知道收益来自 alpha 还是 beta；
- [ ] 费用按真实佣金/印花税/过户费设置；
- [ ] 股票池/市值/流动性过滤与实盘可交易范围一致；
- [ ] 模拟盘运行 ≥20 个交易日，跟踪误差可解释；
- [ ] 有止损/最大回撤/单票上限；
- [ ] 策略代码、参数、数据版本可复现（code_hash/backtest_id）；
- [ ] 已阅读免责声明，不把回测当收益承诺。

附录 A：内置模板与默认策略

- 内置模板约 **95 个**：基础 37、进阶 48、风控 10；
- 目录：A 股策略 01⌧10、minibt 策略（趋势/均值回归/通道/量价）、通用策略；
- 默认策略 13 个，详见 `docs/QuantMind_默认策略手册.md`；
- 模板 JSON 字段：`id/name/description/category/difficulty/params/execution_defaults/live_defaults/live_config_tips/markets/dir`。

模板参数声明示例：

```
{
"name": "topk",
"description": "持仓股票总数",
"default": 50,
"min": 5,
"max": 200
}
```

只有声明在 `params` 里的参数，界面才会渲染为可调项。

附录 B：源码位置（开发者）

主题	文件
策略格式识别	`backend/shared/strategy_format.py`
Qlib 回测请求/结果	`backend/services/engine/qlib_app/schemas/backtest.py`
Qlib 策略类	`backend/services/engine/qlib_app/utils/recording_strategy.py`、`extended_strategies.py`
策略类路由	`backend/services/engine/qlib_app/services/strategy_builder.py`
回测服务	`backend/services/engine/qlib_app/services/backtest_service*.py`
回测持久化	`backend/services/engine/qlib_app/services/backtest_persistence.py`
回测体检	`backend/shared/backtest_health.py`、`backend/scripts/eval/health_check.py`
执行/模拟盘配置	`backend/services/trade_shared/schemas/live_trade_config.py`
模拟盘调度	`backend/services/simulation/services/simulation_hosted_scheduler.py`
minibt 接入	`backend/shared/minibt_qdb.py`、`minibt_result.py`、`minibt_detect.py`
minibt DSL	`minibt/`（vendor 1.3.3）
Strategy Lab SDK	`backend/services/engine/strategy_lab/sdk/context.py`
Strategy Lab 回测	`backend/services/engine/strategy_lab/engine/loop.py`
AI-IDE 入口	`backend/services/engine/routers/ai_ide/executor.py`
沙箱 Context	`backend/services/trade/sandbox/context.py`

附录 C：术语表

术语	说明
`STRATEGY_CONFIG`	平台声明式策略配置，策略库可执行格式
`signal="<PRED>"`	使用模型预测分作为信号
`topk` / `n_drop`	持仓数 / 每期换手数
`rebalance_days`	调仓周期；回测 1⊠60，模拟盘 1/3/5/10/20
`f_*`	基本面硬过滤参数前缀
体检 A/B/L/E	真 alpha / beta 主导 / 运气嫌疑 / 证据不足
PBO	参数扫描过拟合概率
DSR	Deflated Sharpe Ratio
MinTRL	最小可置信样本长度
minibt	脚本策略框架，专用 runner 运行
on_tick	沙箱策略钩子，模拟盘每 10 秒调用

最后提醒：策略代码、参数、数据版本要一起留档；回测不是收益承诺。先在策略实验室/回测中心验证逻辑，再在模拟盘积累证据，最后才谈实盘。